



Facultad de Ingeniería
Ingeniería de Sistemas e Informática y Software

Tema:

“Desarrollo de un Sistema Web Integral para mejorar la gestión administrativa en el Gimnasio STRKE”

Autor(es):

Alabrin Huaman Edson Jair U21304346

Flores Hilares Yimy Jhon U22211273

Montes Cañapataña, Héctor U22249054

Saavedra Guadalupe Yadhira Patrica U22314068

Docente:

Ing. Milla Flores, José Luis

Lima - Perú

02 de junio del 2026

Dedicatoria

A nuestras familias, por su apoyo incondicional y constante motivación durante nuestra formación académica. A todas las personas que creyeron en nuestras capacidades y nos impulsaron a alcanzar nuestras metas.

Los autores

Agradecimientos

El presente trabajo nos gustaría agradecer a nuestro docente; José Luis Milla Flores, por su orientación y enseñanza durante el curso, así como a los integrantes del equipo, por su esfuerzo, dedicación y compromiso académico.

Los autores

Resumen

El presente proyecto tiene como finalidad el desarrollo de un sistema web integral orientado a mejorar la gestión administrativa del gimnasio STRKE. Actualmente, muchos procesos dentro del gimnasio se realizan de manera manual, lo cual genera errores en el registro de clientes, dificultades en el control de membresías, problemas en el seguimiento de asistencia y una limitada organización de los ingresos económicos.

Ante esta problemática, se propone el diseño e implementación de una solución tecnológica que permita automatizar los procesos clave del negocio. El sistema permitirá registrar afiliados, gestionar membresías, controlar la asistencia diaria, generar reportes y facilitar la toma de decisiones mediante información organizada.

La implementación de este sistema no solo contribuirá a mejorar la eficiencia operativa del gimnasio, sino que también optimizará la experiencia del usuario y reducirá los errores humanos. Además, permitirá contar con información centralizada y accesible en tiempo real.

En conclusión, el desarrollo de este sistema web representa una oportunidad para modernizar la gestión administrativa del gimnasio STRKE, incrementando su competitividad y sostenibilidad en el mercado.

Palabras Clave: sistema web, gestión administrativa, gimnasio, control de membresías, control de asistencia, automatización de procesos, toma de decisiones, sistemas de información

Abstract

The purpose of this project is to develop a comprehensive web-based system designed to improve the administrative management of the STRKE gym. Currently, many processes within the gym are performed manually, leading to errors in client registration, difficulties in membership management, problems tracking attendance, and limited organization of revenue.

To address these issues, the design and implementation of a technological solution is proposed to automate key business processes. The system will allow for member registration, membership management, daily attendance tracking, report generation, and facilitate decision-making through organized information.

The implementation of this system will not only contribute to improving the gym's operational efficiency but will also optimize the user experience and reduce human error. Furthermore, it will provide centralized and accessible information in real time.

In conclusion, the development of this web-based system represents an opportunity to modernize the administrative management of the STRKE gym, increasing its competitiveness and sustainability in the market.

Keywords: web system, administrative management, gym, membership control, attendance control, process automation, decision making, information systems

TABLA DE CONTENIDO

Dedicatoria	2
Agradecimientos	3
Resumen	4
Abstract	5
Introducción	8
CAPÍTULO 1.....	9
Fundamentos del proyecto	9
1.1. Descripción del proyecto	9
1.2. Realidad Problemática	9
1.3. Justificación	9
1.4. Objetivos	10
1.4.1. Objetivo General	10
1.4.2. Objetivos Específicos	10
1.5. Alcances y limitaciones	11
1.5.1. Alcances	11
1.5.2. Limitaciones	11
1.6. Requerimientos del sistema	12
1.6.1. Requerimientos funcionales	12
1.6.2. Requerimientos No Funcionales	13
CAPÍTULO 2.....	14
Marco Teórico	14
2.1. Antecedentes	14
2.2. Bases teóricas	15
2.2.1. Sistema Web	15
2.2.2. Sistema de Gestión	16
2.2.3. Tecnologías utilizadas	16
2.2.3.1. Lenguaje de programación	16
2.2.3.2. Framework	16
2.2.3.3. Base de datos	16
2.2.3.4. Tecnologías de interfaz	16
2.2.4. Control de Versiones (Git)	17
2.2.5. Plataforma de Desarrollo Colaborativo (GitHub)	17
a. Git Flow	18

b. Características Git Flow aplicadas	18
• Rama main y develop protegidas	18
• Ramas feature/* para cada tarea	19
• PR de feature a develop con Code Review	19
• Tag en main marcando la versión	20
• Issues cerrados con Closes #N	20
• Tablero Kanban activo.....	20
CAPÍTULO 3.....	21
Metodología.....	21
3.1. Metodología de desarrollo	21
3.2. Uso de GitHub en el proyecto	21
CAPÍTULO 4.....	22
Gestión del proyecto	22
4.1. Creación del repositorio.....	22
4.2. Estructura del proyecto.....	22
4.3. Estrategia de Ramas	23
4.3.1. Rama main	23
4.3.2. Rama develop	23
4.3.3. Ramas feature/*	23
4.4. Gestión de commits.....	24
4.5. Uso de GitHub Projects (Kanban)	24
4.6. Issues y organización.....	25
4.7. Herramienta de gestión de actividades (Jira).....	26
4.8. Integración GitHub y Jira.....	27
4.9. Resolución de Merge Conflict.....	30
4.10. Revisión y aprobación del Pull Request	32
4.11. Herramienta de colaboración (Trello).....	35
4.12. Release y Tag	37
CAPÍTULO 5.....	39
Diagrama del proyecto	39
5.1. Base de Datos.....	39
Conclusiones	40

Introducción

La gestión adecuada de la información es un aspecto fundamental dentro de las organizaciones que brindan servicios, especialmente en el sector deportivo, donde los gimnasios requieren llevar un control constante de usuarios, membresías, pagos y asistencia. En este contexto, muchos gimnasios, en especial los de pequeña y mediana escala, aún enfrentan dificultades debido al uso de métodos manuales o herramientas no integradas, lo que genera errores en los registros, falta de control en tiempo real y procesos poco eficientes que afectan la calidad del servicio.

Ante esta problemática, el presente proyecto propone el desarrollo de un sistema web de gestión integral para el gimnasio STRKE, orientado a optimizar los procesos administrativos mediante la automatización de operaciones como el registro de usuarios, control de membresías, gestión de pagos y seguimiento de la asistencia. El sistema se desarrolla bajo una arquitectura en capas con Spring Boot junto a Thymeleaf y Bootstrap, permitiendo una comunicación eficiente entre sus componentes y facilitando su escalabilidad y mantenimiento.

Para garantizar un desarrollo organizado y colaborativo, el proyecto incorpora un conjunto de herramientas complementarias. Se utilizó GitHub como plataforma de control de versiones, aplicando el flujo de trabajo GitFlow mediante ramas main, develop y feature/, junto con Pull Requests y revisión de código entre los integrantes del equipo. Asimismo, se integró Jira como herramienta de gestión de actividades, permitiendo planificar y dar seguimiento al avance del desarrollo mediante sprints, tableros Kanban y la vinculación directa entre tareas e issues con las ramas y commits del repositorio.

Finalmente, esta solución busca mejorar la eficiencia operativa del gimnasio, reducir errores humanos y centralizar la información, aportando una herramienta tecnológica funcional y adaptable a las necesidades del entorno.

CAPÍTULO 1

Fundamentos del proyecto

1.1. Descripción del proyecto

El proyecto consiste en el desarrollo de un sistema web para mejorar la gestión administrativa para el gimnasio STRKE, orientado a mejorar la administración de usuarios, membresías, pagos y asistencia. La solución busca reemplazar los procesos manuales por un sistema automatizado que permita un mejor control de la información. El sistema se desarrollará utilizando Spring Boot y una base de datos relacional, aplicando buenas prácticas de desarrollo y control de versiones mediante Github.

1.2. Realidad Problemática

Desde el inicio de sus actividades, el gimnasio STRKE ha gestionado sus procesos administrativos de forma manual, lo que ha generado diversas dificultades en el control y organización de la información. Esta forma de trabajo tradicional provoca inconsistencias en el registro de usuarios, membresías, pagos y asistencia, así como errores humanos que afectan la exactitud de los datos.

Como consecuencia, en múltiples ocasiones se presentan desorganización en la gestión de membresías, falta de control en los pagos y dificultades en el seguimiento de la asistencia de los clientes, lo que impacta negativamente en la eficiencia operativa y en la calidad del servicio brindado. Asimismo, el uso de métodos manuales demanda mayor tiempo y esfuerzo por parte del personal, dificultando la toma de decisiones oportunas.

Ante la presente situación, se hace evidente la necesidad de implementar una solución tecnológica que permita automatizar los procesos, mejorar el control de la información y optimizar la gestión del gimnasio

1.3. Justificación

El desarrollo de un sistema web de gestión integral para el gimnasio STRKE contribuye a mejorar los procesos administrativos al automatizar tareas como el registro de usuarios, la gestión de membresías, el control de pagos y el seguimiento de la asistencia. Además,

permite reducir errores manuales, agilizar el trabajo del personal y facilitar una administración más ordenada y precisa de la información.

Este proyecto es relevante porque demuestra cómo la implementación de herramientas tecnológicas puede optimizar el funcionamiento interno de un gimnasio, mejorar la organización de los datos y fortalecer la toma de decisiones. Asimismo, se emplean tecnologías como Spring Boot, base de datos MySQL, Bootstrap y Thymeleaf, las cuales permiten desarrollar una aplicación web funcional, dinámica y accesible para los usuarios. De igual manera, el uso de GitHub facilita la gestión del proyecto mediante control de versiones y trabajo colaborativo.

Por lo tanto, el presente proyecto justifica su desarrollo en la necesidad de contar con una solución tecnológica moderna y funcional que permita gestionar la información de manera confiable, centralizada y accesible, contribuyendo a mejorar la eficiencia operativa y la calidad del servicio brindado a los clientes del gimnasio STRKE.

1.4. Objetivos

1.4.1. Objetivo General

- Desarrollar un sistema web de gestión para el gimnasio STRKE, aplicando herramientas de desarrollo colaborativo como Git, GitHub y control de versiones para garantizar la trazabilidad del proyecto.

1.4.2. Objetivos Específicos

- Gestionar el código fuente del proyecto mediante Git, aplicando control de versiones y manejo de ramas por módulo
- Diseñar e implementar el sistema web usando Spring Boot, Thymeleaf, Bootstrap y MySQL como stack tecnológico.
- Utilizar GitHub como plataforma colaborativa para el seguimiento de cambios, commits y revisión de código del equipo.
- Garantizar la trazabilidad del desarrollo mediante una convención de commits que vincule cada cambio con su funcionalidad correspondiente

1.5. Alcances y limitaciones

1.5.1. Alcances

El presente proyecto comprende el desarrollo de un sistema web de gestión integral para el gimnasio STRKE, orientado a mejorar la administración de sus procesos internos mediante una solución digital. El sistema contempla las siguientes funcionalidades principales:

Módulo de autenticación: permite el inicio de sesión de usuarios mediante credenciales, asegurando el acceso al panel administrativo.

Módulo de dashboard: muestra información general del sistema, como cantidad de afiliados, ingresos y estado operativo, facilitando una visión rápida del negocio.

Módulo de afiliados: permite registrar, modificar, eliminar y consultar información de los clientes, incluyendo datos personales y estado de membresía.

Módulo de asistencia: permite registrar y consultar la asistencia de los afiliados, facilitando el control de ingreso al gimnasio.

Módulo de membresías: permite asignar, gestionar y visualizar membresías activas, incluyendo duración, precio y estado.

Configuración de membresías: permite crear, editar y eliminar tipos de membresías según las necesidades del gimnasio.

Interfaz web intuitiva: el sistema contará con una interfaz amigable desarrollada con Bootstrap y Thymeleaf, facilitando su uso por parte del personal administrativo.

1.5.2. Limitaciones

Limitación por tiempo de desarrollo

Debido al tiempo reducido del proyecto, no será posible implementar funcionalidades avanzadas, centrándose únicamente en los módulos principales como afiliados, membresías, pagos y asistencia.

Limitación tecnológica

El sistema se desarrollará como una aplicación web utilizando Spring Boot, Thymeleaf y Bootstrap, sin incluir versión móvil o aplicación nativa.

Limitación de recursos

No se integrarán tecnologías adicionales como sistemas biométricos o dispositivos automatizados para el control de acceso, debido a la disponibilidad limitada de recursos.

Limitación en infraestructura

La base de datos será gestionada en un entorno local mediante MySQL, sin implementación en servidores en la nube ni acceso remoto, lo que limita su escalabilidad.

Limitación de alcance funcional

El sistema está enfocado a la gestión de un solo gimnasio y no contempla funcionalidades avanzadas como análisis de datos, integración con sistemas externos o automatización compleja.

1.6. Requerimientos del sistema

1.6.1. Requerimientos funcionales

Los requerimientos funcionales describen las funcionalidades que el sistema debe cumplir para satisfacer las necesidades del gimnasio STRKE. Estos se derivan directamente de la problemática identificada y de los procesos del negocio.

Lista de requerimientos funcionales

- **RF01:** Iniciar sesión en el sistema
- **RF02:** Registrar afiliados
- **RF03:** Editar afiliados
- **RF04:** Eliminar afiliados
- **RF05:** Consultar afiliados
- **RF06:** Gestionar membresías

- **RF07:** Asignar membresías
- **RF08:** Registrar pagos
- **RF09:** Consultar historial de pagos
- **RF10:** Registrar asistencia
- **RF11:** Consultar asistencia
- **RF12:** Visualizar Dashboard

Estos requerimientos permiten automatizar los procesos manuales actuales, mejorando la organización de la información, reduciendo errores humanos y facilitando la gestión administrativa del gimnasio.

1.6.2. Requerimientos No Funcionales

Los requerimientos no funcionales establecen las características de calidad que debe cumplir el sistema para garantizar su correcto funcionamiento, rendimiento y usabilidad.

Lista de requerimientos no funcionales

- **RNF01:** Interfaz intuitiva y amigable (Bootstrap)
- **RNF02:** Acceso seguro mediante autenticación
- **RNF03:** Tiempo de respuesta menor a 5 segundos
- **RNF04:** Disponibilidad en horario operativo
- **RNF05:** Arquitectura en capas con Spring Boot
- **RNF06:** Escalabilidad para futuras mejoras
- **RNF07:** Compatibilidad con navegadores modernos
- **RNF08:** Integridad de datos en MySQL

CAPÍTULO 2

Marco Teórico

2.1. Antecedentes

Para la realización de este proyecto, se tomaron en consideración diversos trabajos previos relacionados con el desarrollo de sistema web de gestión, tales como tesis y proyectos de pregrado que abordan temáticas similares al objetivo del estudio. Estas investigaciones permitieron conocer diferentes enfoques y soluciones aplicadas en la administración de usuarios, control de membresías, gestión de pagos y seguimiento de asistencia en entorno como gimnasios y otros servicios.

A continuación, se mencionan estudios que han servido como referencia y que aportan antecedentes para la propuesta de Desarrollo de un Sistema Web Integral para mejorar la gestión administrativa en el gimnasio STRKE.

En el contexto internacional; en la tesis titulada “Aplicación web para mejorar la gestión administrativa del departamento de vinculación de la universidad estatal de bolívar” publicado por la Universidad Regional Autónoma de los Andes, Ambato-Ecuador, 2015 tuvo como finalidad mejorar la gestión de programas y proyectos mediante la implementación de un sistema informático que permita organizar, evaluar y monitorear la información. El sistema permitió optimizar los procesos y mejorar el acceso a la información, facilitando la toma de decisiones dentro de la institución. Pachala Guzmán, F. P. (2015). *Sistema de gestión de proyectos de vinculación* [Tesis de pregrado, Universidad Regional Autónoma de los Andes]. Repositorio Institucional UNIANDES.

En el ámbito nacional, en la tesis titulada “Implementación de un Sistema Web para mejorar la gestión de ventas, en la empresa Easy Construction Cr S.A.C” publicado por la Universidad Tecnológica del Perú, Lima-Perú, 2025 tuvo como finalidad optimizar el proceso de ventas mediante la automatización de tareas clave, como la gestión de clientes, control de inventario y registro de operaciones, a través de un sistema web. El sistema fue implementado utilizando tecnologías como Angular como framework de desarrollo, TypeScript como lenguaje de programación y MongoDB como base de datos. Uceda Gutiérrez, J. M. A. (2025). *Implementación de un sistema web para mejorar la gestión*

de ventas en la empresa Easy Construction Cr S.A.C. [Tesis de pregrado, Universidad Tecnológica del Perú]. Repositorio Institucional UTP.

2.2. Bases teóricas

El uso de la tecnología para apoyar los procesos de gestión administrativa se ha convertido en una necesidad fundamental para organizaciones de cualquier tamaño. En el caso de los gimnasios, el control de usuarios, membresías, pagos y asistencia es clave para mantener un adecuado funcionamiento del negocio.

Un manejo ineficiente de estos procesos puede generar desorganización, pérdida de información, errores en registros y dificultades en la atención al cliente. Por ello, los sistemas web se han consolidado como herramientas que permiten optimizar la gestión, mejorar la precisión de los datos y facilitar la toma de decisiones.

A partir de este contexto, el presente marco teórico expone los conceptos fundamentales que sustentan el desarrollo del sistema web de gestión integral para el gimnasio STRKE, así como las tecnologías empleadas para su implementación.

2.2.1. Sistema Web

“Un sistema web es una aplicación que se ejecuta en un navegador y permite a los usuarios acceder, procesar y gestionar información a través de internet o una red interna” (Crea System, 2022). Su propósito es facilitar el acceso a la información, optimizar procesos y mejorar la interacción entre el usuario y el sistema. En el contexto de este proyecto, el sistema web permitirá gestionar la información del gimnasio STRKE, incluyendo afiliados, membresías, pagos y control de asistencia, brindando datos organizados y accesibles para una mejor administración.

Componentes del sistema web aplicado en el proyecto:

- **Software:** aplicación web desarrollada con Spring Boot.
- **Base de datos:** MySQL para almacenar la información de forma estructurada.
- **Usuarios:** administrador y/o personal autorizado del gimnasio.
- **Procesos:** registro, actualización y consulta de afiliados, membresías, pagos y asistencia.

2.2.2. Sistema de Gestión

“Un sistema de gestión es un conjunto de herramientas que permite administrar información y procesos dentro de una organización”. Su propósito es mejorar la eficiencia y organización de las actividades. En este proyecto, el sistema de gestión permitirá organizar y controlar las operaciones del gimnasio STRKE.

2.2.3. Tecnologías utilizadas

2.2.3.1. Lenguaje de programación

Java: Java es un lenguaje orientado a objetos que permite desarrollar aplicaciones complejas y portables. Este proyecto se utiliza para construir la interfaz gráfica y la lógica del sistema, permitiendo la interacción entre el usuario y la base de datos

2.2.3.2. Framework

Spring Boot: Es un framework de código abierto que da soporte para el desarrollo de aplicaciones y páginas webs basadas en Java.

2.2.3.3. Base de datos

MySQL: MySQL Workbench es una interfaz gráfica de usuario (GUI) o herramienta de gestión gráfica que permite interactuar con la base de datos de MySQL Server, asimismo es utilizado para almacenar toda la información. Se eligió MySQL porque es una base de datos eficiente, segura, gratuita y ampliamente utilizada en entornos empresariales. Permite manejar grandes volúmenes de información sin comprometer la velocidad del sistema.

2.2.3.4. Tecnologías de interfaz

Thymeleaf: Es un moderno motor de plantillas Java del lado del servidor, apto tanto para entornos web como para entornos independientes.

Bootstrap: Bootstrap es un framework CSS de código abierto orientado a la creación de interfaces de usuario para la web.

2.2.4. Control de Versiones (Git)

“Es un sistema de control de versiones que permite a los desarrolladores registrar y gestionar los cambios realizados en el código, facilitando el trabajo colaborativo en proyectos” (The Forage, 2024). Su propósito es mantener un historial de versiones, permitiendo recuperar cambios anteriores y organizar el desarrollo de software. En el contexto de este proyecto, Git será utilizado para controlar las versiones del sistema web del gimnasio STRKE, registrando los avances realizados durante su desarrollo y permitiendo una mejor organización del código

Componentes aplicados en el proyecto:

- **Repositorio:** almacenamiento del código del sistema.
- **Commits:** registro de cambios realizados.
- **Ramas:** desarrollo de funcionalidades de manera independiente.
- **Historial:** seguimiento de versiones del proyecto.

2.2.5. Plataforma de Desarrollo Colaborativo (GitHub)

“GitHub es una plataforma de desarrollo colaborativo que aloja proyectos en la nube utilizando el sistema de control de versiones Git, permitiendo a los desarrolladores almacenar, administrar y llevar un registro de los cambios en el código” (EBAC, 2023). Su propósito es facilitar el trabajo en equipo, el seguimiento del progreso y la colaboración entre desarrolladores. En el contexto de este proyecto, GitHub será utilizado para almacenar el código fuente del sistema web del gimnasio STRKE, gestionar los cambios realizados y organizar el desarrollo mediante el uso de repositorios y ramas.

Componentes aplicados en el proyecto:

- **Repositorio:** almacenamiento del código del sistema en la nube.
- **Commits:** registro de cambios realizados en el proyecto.
- **Ramas:** desarrollo de funcionalidades de forma independiente.
- **Issues:** seguimiento de tareas y problemas.

a. Git Flow

Es un modelo de trabajo para Git, creado por Vincent Driessen en 2010, que estructura la gestión de ramas para proyectos de software. Utiliza ramas dedicadas (main, develop, feature, release, hotfix) para organizar el desarrollo, permitiendo trabajar en paralelo y gestionar lanzamientos de forma ordenada y profesional.

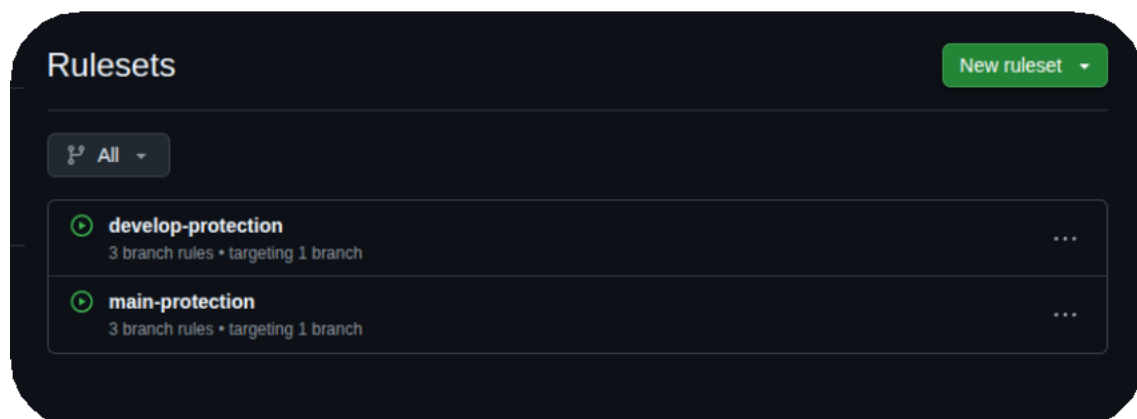


Aplicamos un Git Flow completo:

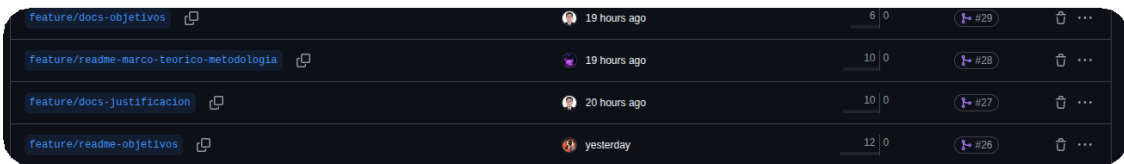
feature/* → develop → release/v1.0-entrega-parcial-1 → main
tag: v1.0-entrega-parcial-1

b. Características Git Flow aplicadas

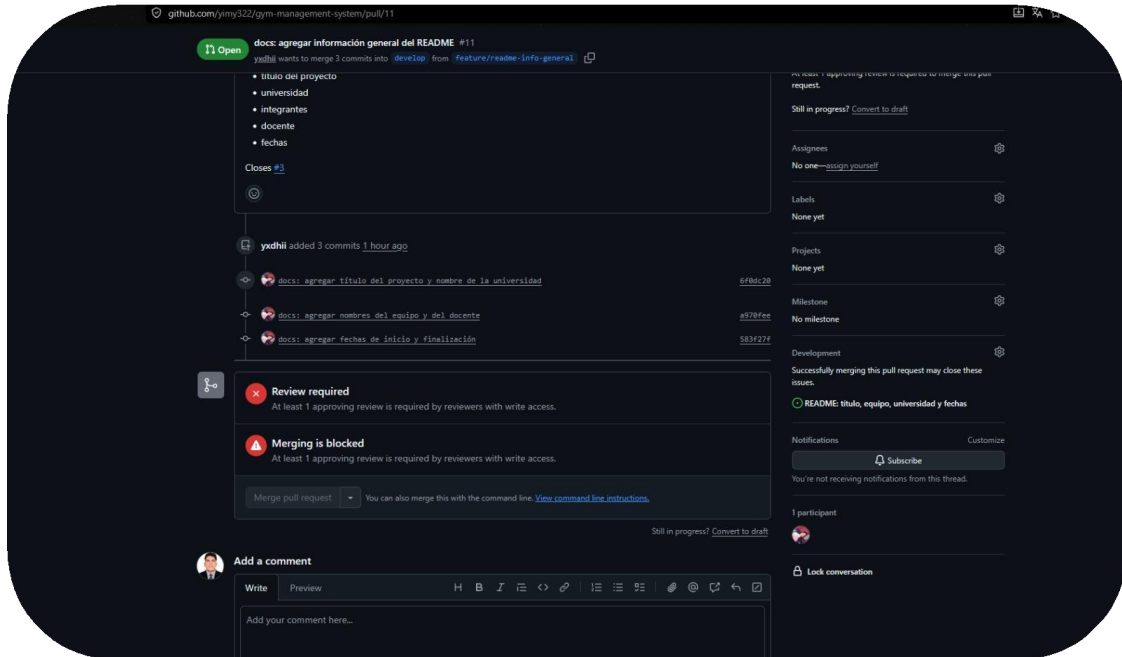
- *Rama main y develop protegidas*



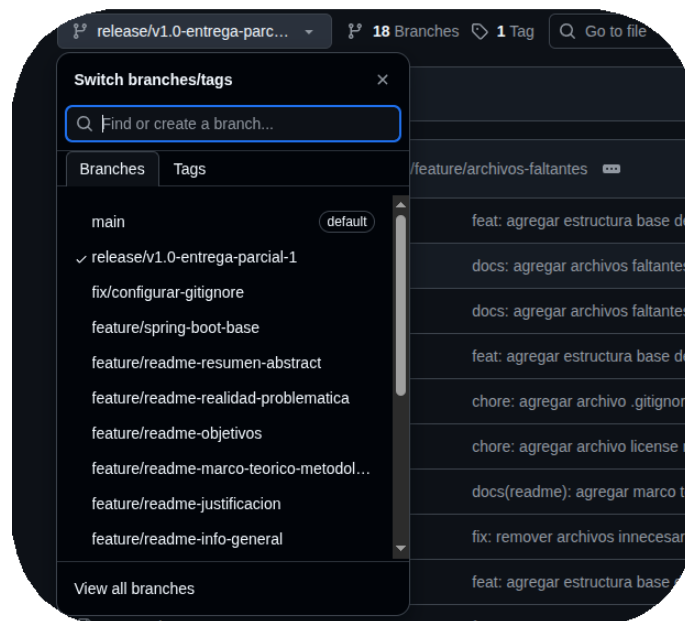
- *Ramas feature/* para cada tarea*



- *PR de feature a develop con Code Review*



- *Rama release para preparar la entrega*



- *Tag en main marcando la versión*

```
yiny@PC-322:~/universidad/ciclo-8-prt2/Herramientas-desarrollo/gym-management-system$ git tag -a v1.0-entrega-parcial-1 -m "Avance 1 completo"
yiny@PC-322:~/universidad/ciclo-8-prt2/Herramientas-desarrollo/gym-management-system$ git push origin v1.0-entrega-parcial-1
Username for 'https://github.com': yiny322
Password for 'https://yiny322@github.com':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 171 bytes | 171.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com:yiny322/gym-management-system.git
 * [new tag]          v1.0-entrega-parcial-1 -> v1.0-entrega-parcial-1
```

- *Issues cerrados con Closes #N*

docs(readme): agregar justificacion (Closes #9)

 yxdhii created `feature/readme-justificacion` • bd64e19 • 21 hours ago

- *Tablero Kanban activo*

Gym Management System

View 1 + New view

Filter by keyword or by field

Todo 0	In Progress 0	Done 16
This item hasn't been started	This is actively being worked on	This has been completed
		- gym-management-system #1 Agregar archivo .gitignore - gym-management-system #2 Agregar archivo LICENSE - gym-management-system #4 Subir cascarón de Spring Boot - gym-management-system #3 Proteger ramas main y develop - gym-management-system #18 Remover archivos innecesarios - gym-management-system #9 README: justificación - gym-management-system #10 README: objetivos generales y específicos - gym-management-system #11 README: marco teórico y metodología

CAPÍTULO 3

Metodología

3.1. Metodología de desarrollo

El desarrollo del sistema se realizará de manera incremental, permitiendo avanzar por módulos (afiliados, membresías, pagos, asistencia). Este enfoque facilita la validación continua de funcionalidades y la mejora progresiva del sistema.

3.2. Uso de GitHub en el proyecto

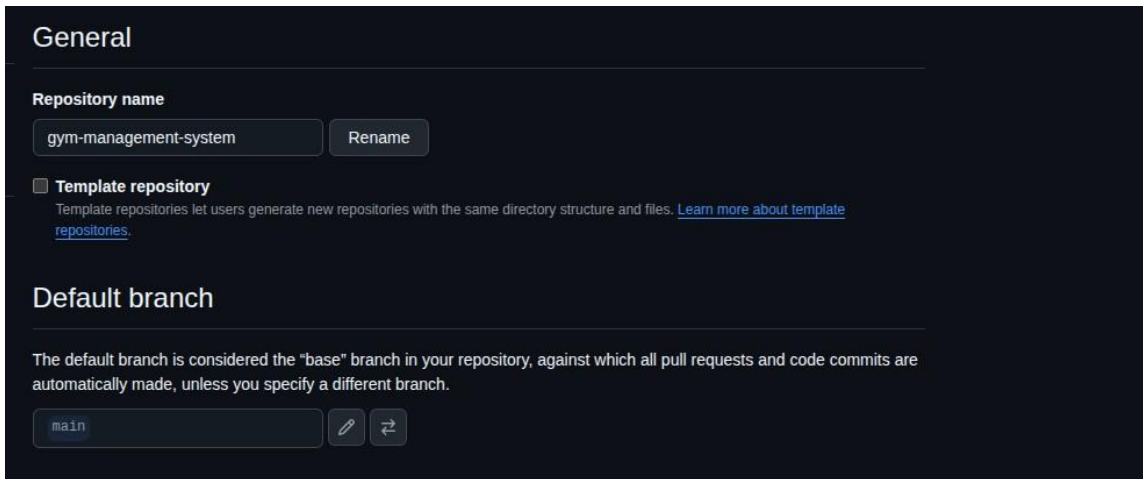
Se utilizará GitHub como herramienta de control de versiones para gestionar el código fuente, mediante el uso de repositorios, ramas y commits. Esto permitirá organizar el trabajo, registrar avances y facilitar el desarrollo colaborativo.

CAPÍTULO 4

Gestión del proyecto

4.1. Creación del repositorio

Se creó un repositorio en GitHub para almacenar el código fuente y la documentación del sistema web de gestión integral para el gimnasio STRKE. Este repositorio permitirá centralizar el desarrollo del proyecto, registrar los avances y mantener un historial de cambios.



The screenshot shows the 'General' tab of a new GitHub repository. The repository name is 'gym-management-system'. There is a 'Rename' button. The 'Template repository' checkbox is unchecked. Below it, a note says 'Template repositories let users generate new repositories with the same directory structure and files. [Learn more about template repositories.](#)'. The 'Default branch' section explains that the default branch is the 'base' branch for pull requests and commits. The default branch is set to 'main'.

General

Repository name

gym-management-system Rename

☐ **Template repository**

Template repositories let users generate new repositories with the same directory structure and files. [Learn more about template repositories.](#)

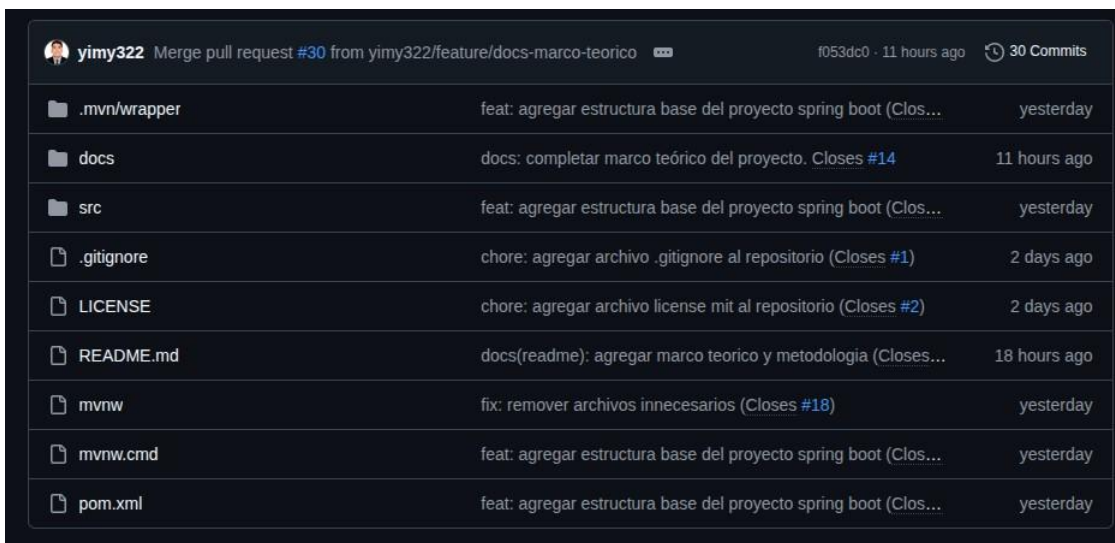
Default branch

The default branch is considered the "base" branch in your repository, against which all pull requests and code commits are automatically made, unless you specify a different branch.

main edit ↺

4.2. Estructura del proyecto

El repositorio se organizó en carpetas para separar el código fuente, la documentación y los informes del proyecto. Esta estructura permite mantener un orden adecuado y facilitar la ubicación de los archivos.



The screenshot shows a table of commit history. The header includes the user 'yimy322', a merge pull request #30, the commit hash 'f053dc0', the time '11 hours ago', and '30 Commits'. The table lists various files and folders with their commit messages and timestamps.

File/Folder	Commit Message	Time
.mvn/wrapper	feat: agregar estructura base del proyecto spring boot (Closes...	yesterday
docs	docs: completar marco teórico del proyecto. Closes #14	11 hours ago
src	feat: agregar estructura base del proyecto spring boot (Closes...	yesterday
.gitignore	chore: agregar archivo .gitignore al repositorio (Closes #1)	2 days ago
LICENSE	chore: agregar archivo license mit al repositorio (Closes #2)	2 days ago
README.md	docs(readme): agregar marco teorico y metodologia (Closes...	18 hours ago
mvnw	fix: remover archivos innecesarios (Closes #18)	yesterday
mvnw.cmd	feat: agregar estructura base del proyecto spring boot (Closes...	yesterday
pom.xml	feat: agregar estructura base del proyecto spring boot (Closes...	yesterday

4.3. Estrategia de Ramas

Para el desarrollo del proyecto se utilizará una estrategia de ramas que permita organizar el trabajo y evitar modificaciones directas en la rama principal.

4.3.1. Rama main

La rama **main** será utilizada como rama principal del proyecto, donde se mantendrá la versión estable del sistema.

4.3.2 Rama develop

La rama **develop** será utilizada para integrar los avances del desarrollo antes de ser enviados a la rama principal.

4.3.3 Ramas feature/*

Las **ramas feature/*** se utilizarán para desarrollar funcionalidades específicas del sistema, como afiliados, membresías, pagos o asistencia.

Default					
Branch	Updated	Check status	Behind	Ahead	Pull request
main	4 days ago				Default
Your branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
feature/archivos-faltantes	9 hours ago		0	29	
develop	11 hours ago		0	29	
feature/spring-boot-base	yesterday		0	6	#17
feature/add-gitignore	2 days ago		0	3	#16
feature/add-license	2 days ago		0	1	
Active branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
feature/archivos-faltantes	9 hours ago		0	29	
develop	11 hours ago		0	29	
feature/docs-marco-teorico	11 hours ago		0	28	#30
feature/docs-objetivos	17 hours ago		0	26	#29
feature/readme-marco-teorico-metodologia	18 hours ago		0	22	#28
View more branches >					

4.4. Gestión de commits

Los commits se realizarán de manera periódica para registrar los avances del proyecto. Cada commit tendrá un mensaje descriptivo que permita identificar el cambio realizado.

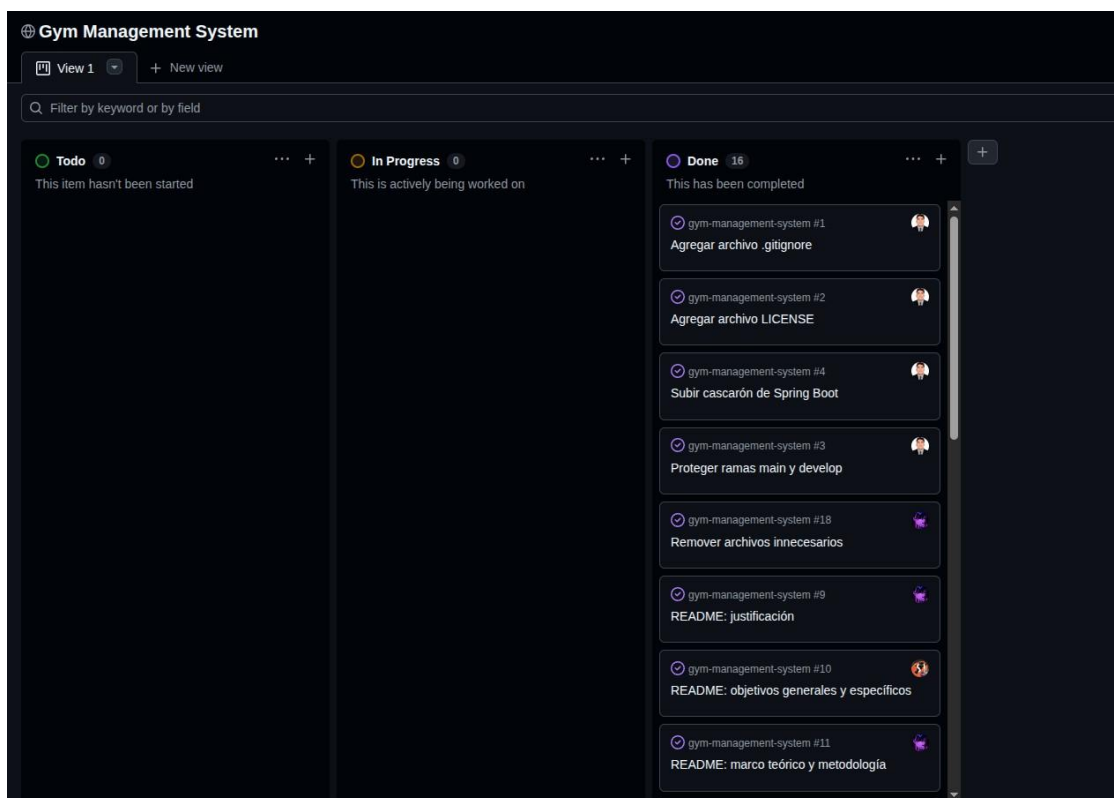
Ejemplos:

4.5 Uso de GitHub Projects (Kanban)

Se utilizará GitHub Projects como tablero Kanban para organizar las actividades del proyecto. Las tareas se clasificarán según su estado: pendientes, en proceso y finalizadas.

Estados del tablero:

- Pendiente
- En proceso
- Finalizado

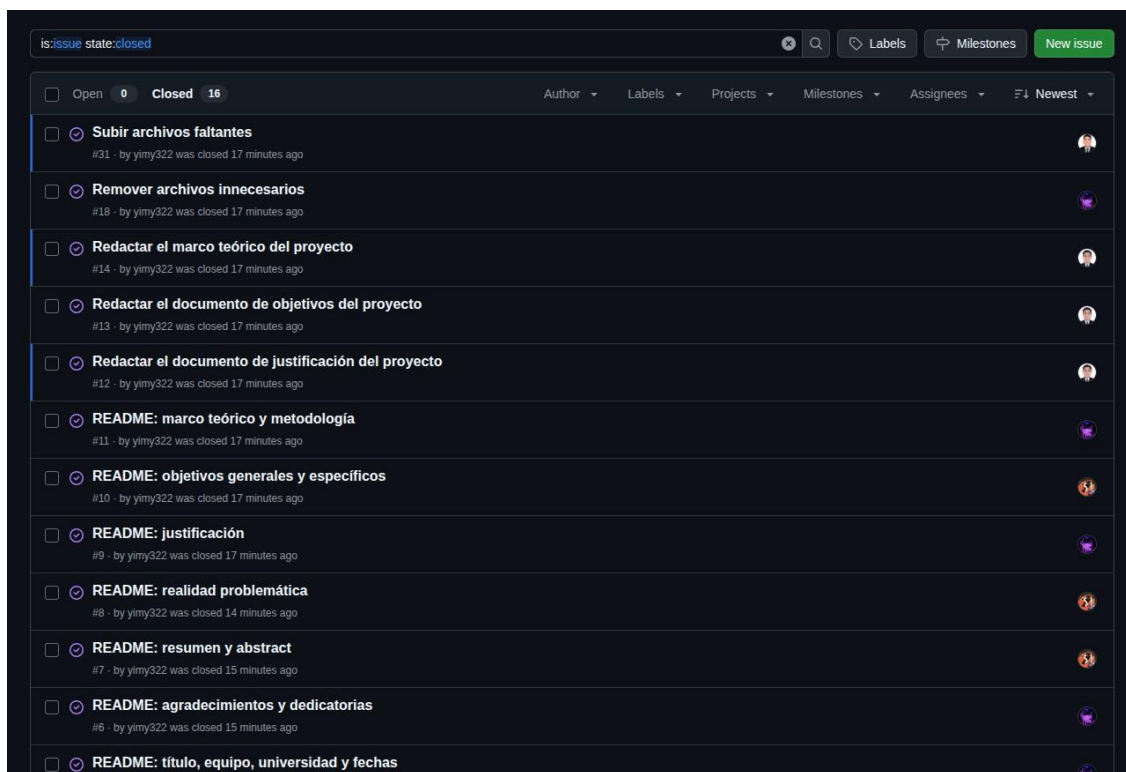


4.6. Issues y organización

Los Issues se utilizarán para registrar tareas, mejoras o problemas relacionados con el desarrollo del sistema. Cada issue permitirá dar seguimiento a una actividad específica y podrá vincularse con commits o ramas del proyecto.

Ejemplos de Issues:

- Proteger ramas main y develop
- Agregar archivo LICENSE
- README: marco teórico y metodología
- Elaborar documentación inicial
- Actualizar README.md

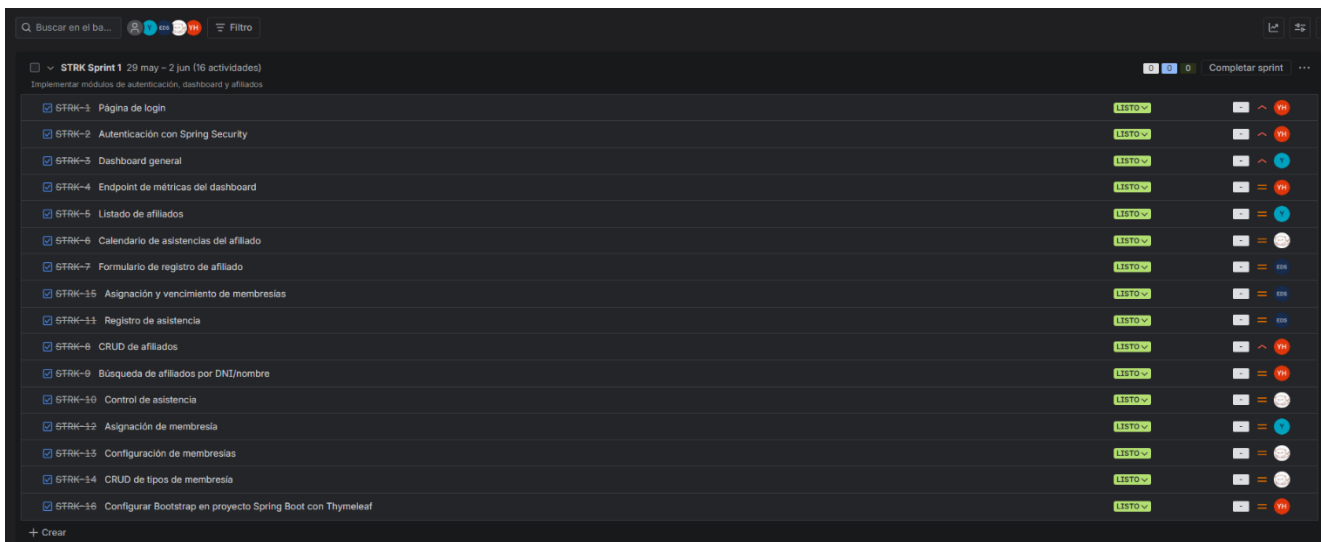


4.7. Herramienta de gestión de actividades (Jira)

Jira es una plataforma de gestión de proyectos desarrollada por Atlassian que permite planificar, rastrear y organizar el trabajo del equipo mediante tickets, sprints y tableros visuales.

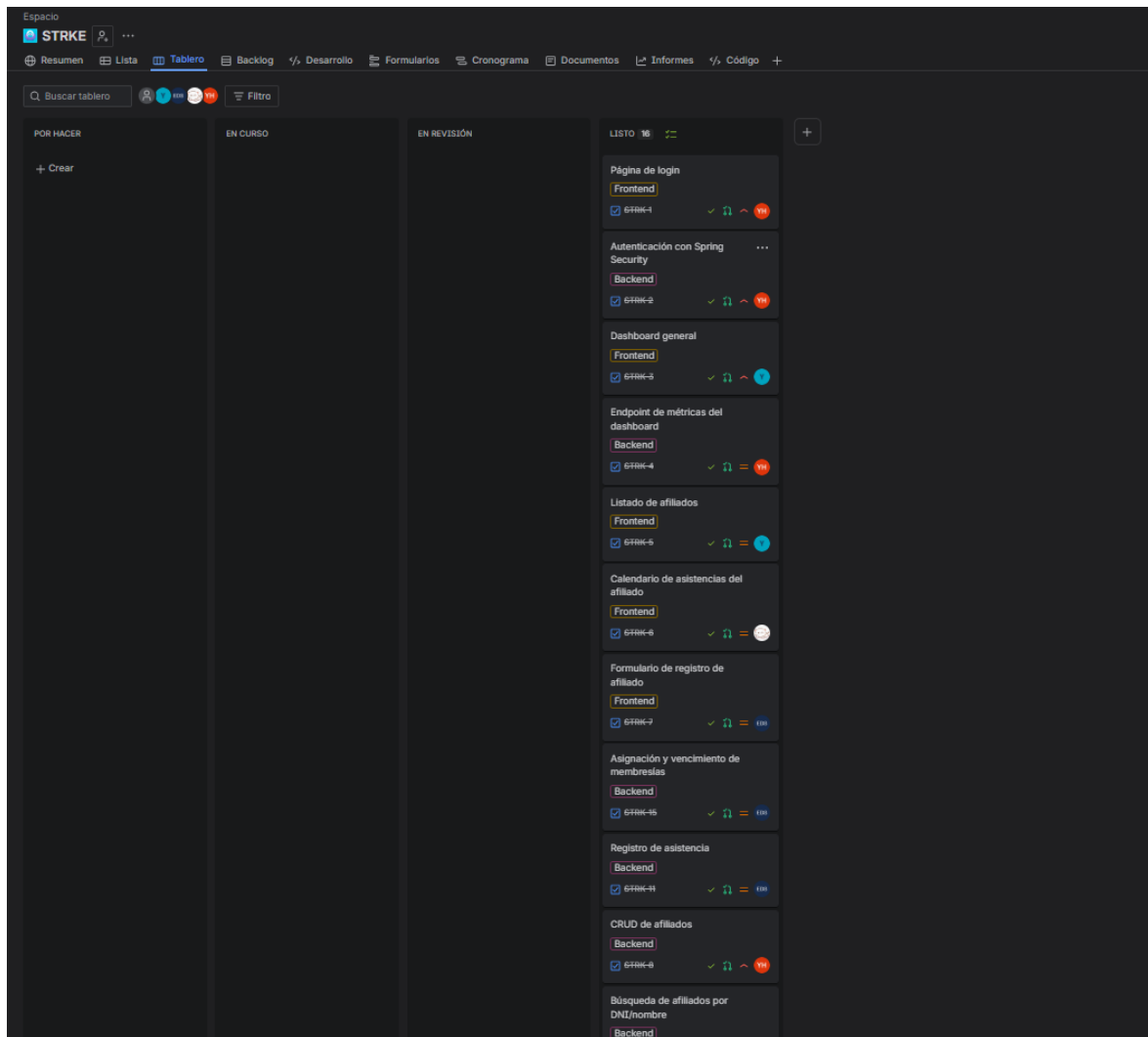
Sprint 1

Se creó el **STRK Sprint 1** con un total de 16 actividades orientadas a implementar los módulos de autenticación, Dashboard y afiliados. Al finalizar el sprint, las 16 tareas se encuentran en estado **Listo**.



Tablero Kanban

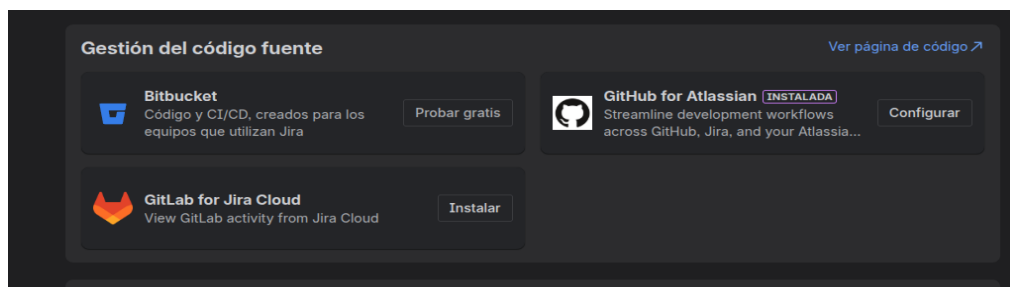
El tablero organiza las tareas en cuatro columnas según su estado: **Por hacer**, **En curso**, **En revisión** y **Listo**. Permite visualizar el avance del sprint en tiempo real identificando qué tareas están pendientes y cuáles fueron completadas.



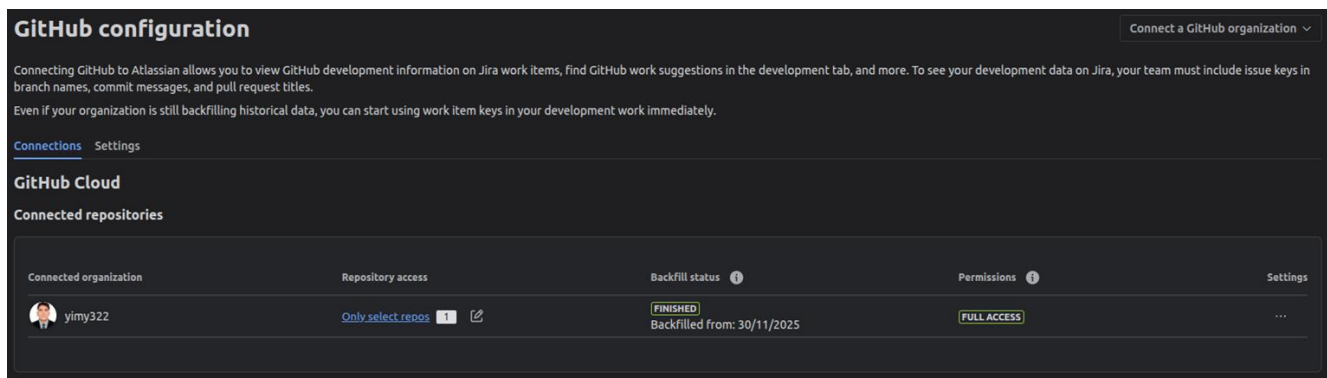
4.8. Integración GitHub y Jira

Para vincular el seguimiento de tareas con el desarrollo del código, se integró Jira con GitHub mediante el plugin **GitHub for Atlassian**.

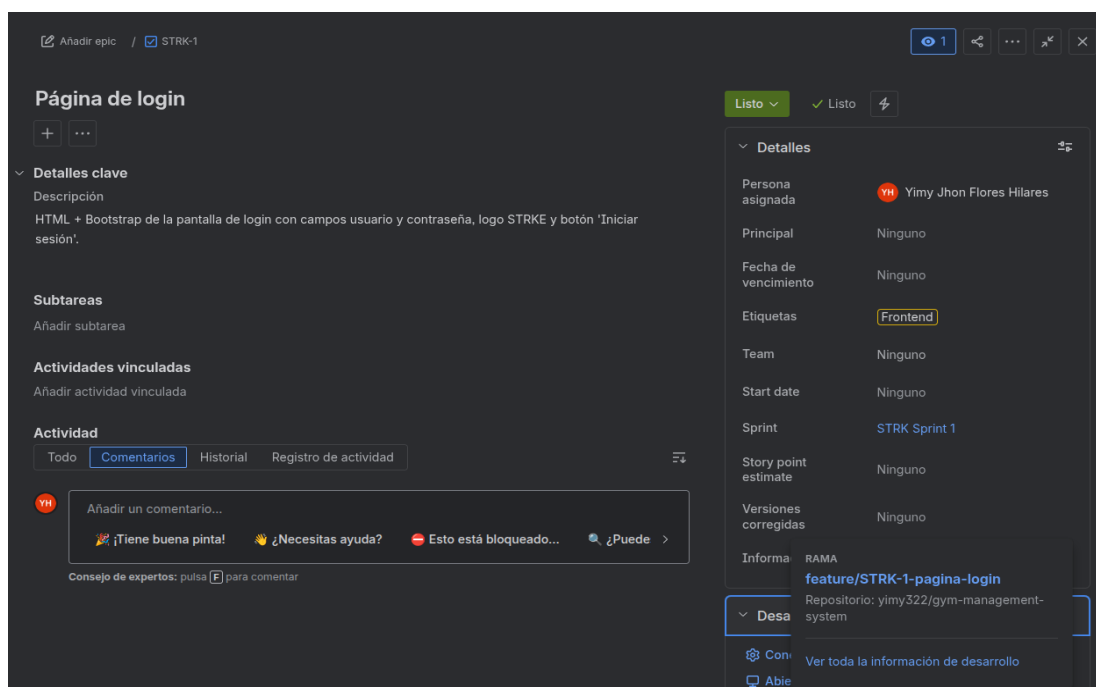
Paso 1. Instalación del plugin de GitHub for Atlassian en Jira.



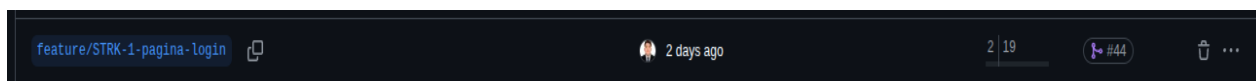
Paso 2. Conectar la cuenta de GitHub y seleccionar los repositorios a vincular.



Paso 3. Usar el identificador del issue (ej. STRK-1) en el nombre de las ramas y commits para vincularlos automáticamente con su tarea en Jira.



Rama: *feature/STRK-1-pagina-login* vista desde el github



Rama: *feature/STRK-1-pagina-login* ya vinculada en el issue correspondiente gracias al identificador **STRK-1**

Desarrollo de STRK-1

Dar feedback

Rama
Confirmacion
Solicitudes de incorporación de ca
Compilacion
Implementacion
Indicadores de funcionalid
Otros enlac

yimy322/gym-management-system (GitHub)

Rama	Solicitud de incorporación de cambios	Acción
feature/STRK-1-pagina-login	1 Solicitudes de incorporación de cambios 1 FUSIONADA	Crear solicitud de incorporación de cambios

Commit: feat: maquetar pagina de login con bootstrap - STRK-1 #done

Activity

feature/STRK-1-...
All activity
All users
All time

feat: maquetar pagina de login con bootstrap - STRK-1 #done
yimy322 pushed 1 commit • 4a70174...ace59ca • yesterday

Paso 4. Configuramos una regla de automatización en Jira para cerrar el issue automáticamente cuando un commit incluye #done en su mensaje.

Cerrar issue con #done

Commit creado
El flujo se ejecuta al crear una confirmación

Se ha aplicado la condición
{{(commit.message)} contiene #done

Realizar la transición de la actividad a LISTO

Detalles del flujo
Los campos obligatorios están marcados con un asterisco *

Nombre * (requerido)
Cerrar issue con #done

Descripción
Añadir una descripción a tu flujo

Alcance
STRKE
Solo se puede modificar el alcance en la administración global.

Propietario * (requerido)
Yimy Jhon Flores Hilares

El propietario recibirá mensajes de correo electrónico el flujo genere un error.

Agente * (requerido)
Automation for Jira

Las acciones definidas en este flujo las realizará el usuario seleccionado como actor. Obtén más información sobre los actores de flujo en la automatización.

Notificar al producirse un error
Enviar un mensaje de correo electrónico al propietario del flujo una...

¿Quién puede editar este flujo? * (requerido)
Privado

☐ Marca esto para permitir que las acciones de flujo desencadenen

***** Cada issue muestra su rama y repositorio vinculado directamente desde Jira.***

4.9. Resolución de Merge Conflict

Durante el desarrollo, al intentar fusionar la rama develop con feature/STRK-2-spring-security se generó un conflicto en los archivos modificados por ambas ramas simultáneamente. A continuación, se describe el proceso de resolución.

Paso 1. Conflicto detectado

Al ejecutar git merge develop desde la rama feature/STRK-2-spring-security, Git detectó un conflicto en el archivo src/main/resources/templates/fragments/navbar.html, impidiendo la fusión automática.

```
yiny@PC-322: ~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git checkout feature/STRK-2-spring-security
Switched to branch 'feature/STRK-2-spring-security'
Your branch is up to date with 'origin/feature/STRK-2-spring-security'.
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git merge develop
Auto-merging src/main/resources/templates/fragments/navbar.html
CONFLICT (content): Merge conflict in src/main/resources/templates/fragments/navbar.html
Automatic merge failed; fix conflicts and then commit the result.
yiny@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$
```

Paso 2. Identificación del archivo

VS Code marcó las diferencias entre ambas ramas en el archivo navbar.html: la rama actual (HEAD) contenía el botón de logout con formulario y token de seguridad, mientras que develop tenía un enlace simple sin formulario.

```
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
<<<<<< HEAD (Current Change)
|
|<form th:action="@{/logout}" method="post">
|  <input type="hidden" th:name="{ csrf.parameterName}" th:value="{ csrf.token}"/>
|  <button type="submit" class="btn btn-sm btn-outline-secondary rounded-circle p-1" style="width:32px; height:32px;">
|    <i class="fa-solid fa-arrow-right-from-bracket" style="font-size:12px;"></i>
|  </button>
|</form>
|
|=====
|
|<a class="btn btn-danger" style="width:32px; height:32px;">
|  <i class="fa-solid fa-arrow-right-from-bracket" style="font-size:12px;"></i>
|</a>
|>>>>>> develop (Incoming Change)
|
|</div>
```

Paso 3. Resolución del conflicto

Se conservó el cambio de la rama actual, manteniendo el botón de logout con su formulario de seguridad y eliminando el código proveniente de develop.

```

<form th:action="@{/logout}" method="post">
  <input type="hidden" th:name="${_csrf.parameterName}" th:value="${_csrf.token}"/>
  <button type="submit" class="btn btn-sm btn-outline-secondary rounded-circle p-1" style="width:32px; height:32px;">
    <i class="fa-solid fa-arrow-right-from-bracket" style="font-size:12px;"></i>
  </button>
</form>

```

Paso 4. Commit y push

Se agregó el archivo corregido, se realizó un commit indicando la resolución del conflicto y se subió el cambio a la rama remota.

```

Yadhira Saavedra@LAPTOP-22GAEESQ MINGW64 ~/Documents/STRK
$ git add .
$ git commit -m "fix: resolucion conflicto navbar logout"
$ git push origin feature/STRK-2-spring-security

```

```

yiny@PC-322: ~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git add .
yiny@PC-322: ~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git commit -m "fix: resolucion conflicto navbar logout"
[feature/STRK-2-spring-security 065c484] fix: resolucion conflicto navbar logout
yiny@PC-322: ~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git push origin feature/STRK-2-spring-security
Username for 'https://github.com': yiny322
Password for 'https://yiny322@github.com':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 227 bytes | 227.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/yiny322/gym-management-system.git
 9a6a2d8..065c484 feature/STRK-2-spring-security -> feature/STRK-2-spring-security
yiny@PC-322: ~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$

```

Paso 5. Pull Request sin conflictos

Con el conflicto resuelto, se creó el Pull Request de feature/STRK-2-spring-security hacia develop, quedando a la espera de aprobación para su fusión.

Feature/strk 2 spring security #53 Awaiting approval Code

Open yimy322 wants to merge 2 commits into `develop` from `feature/STRK-2-spring-security`

Conversation 0 Commits 2 Checks 0 Files changed 8 +138 -7

yimy322 commented now Owner

No description provided.

yimy322 added 2 commits 3 hours ago

- feat: configurar spring security y manejo de sesion - STRK-2 #done 9a6a2d8
- fix: resolucion conflicto navbar logout 065c484

Review required
At least 1 approving review is required by reviewers with write access.

Merging is blocked
At least 1 approving review is required by reviewers with write access.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Reviewers

Suggestions

yxdhil Request

At least 1 approving review is required to merge this pull request.

Still in progress? [Convert to draft](#)

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

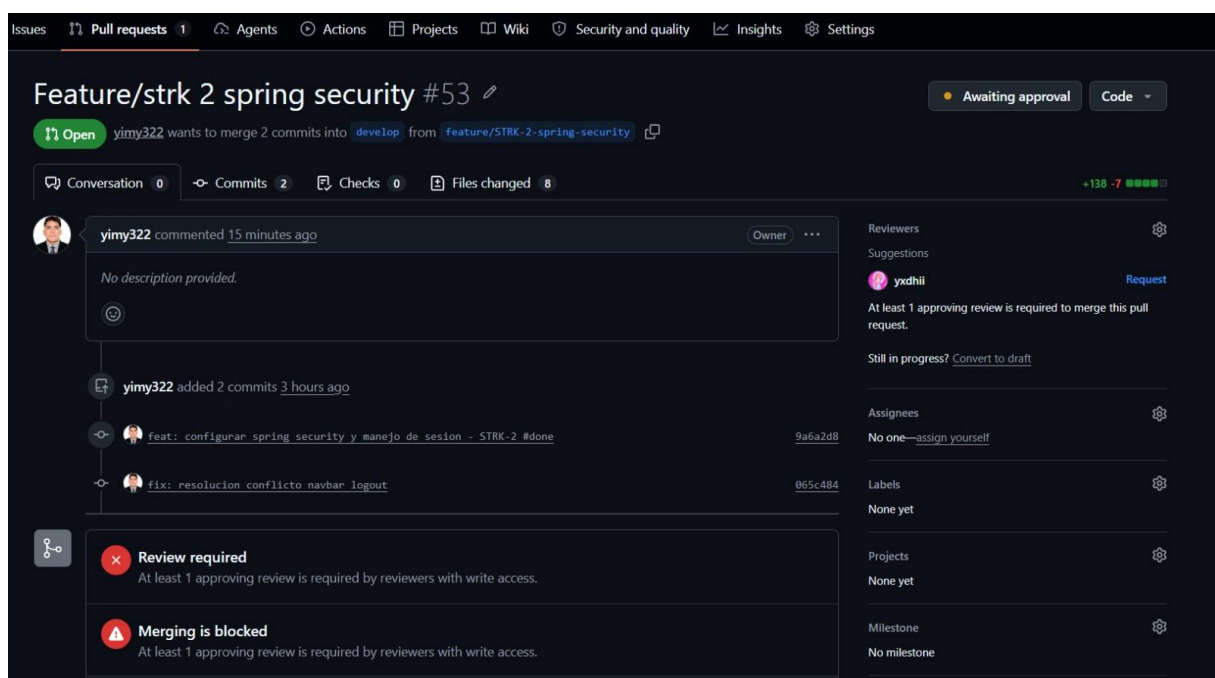
Still in progress? [Convert to draft](#)

4.10. Revisión y aprobación del Pull Request

Para garantizar la calidad del código, se estableció un flujo de revisión donde cada Pull Request requiere la aprobación de al menos un integrante del equipo antes de fusionarse.

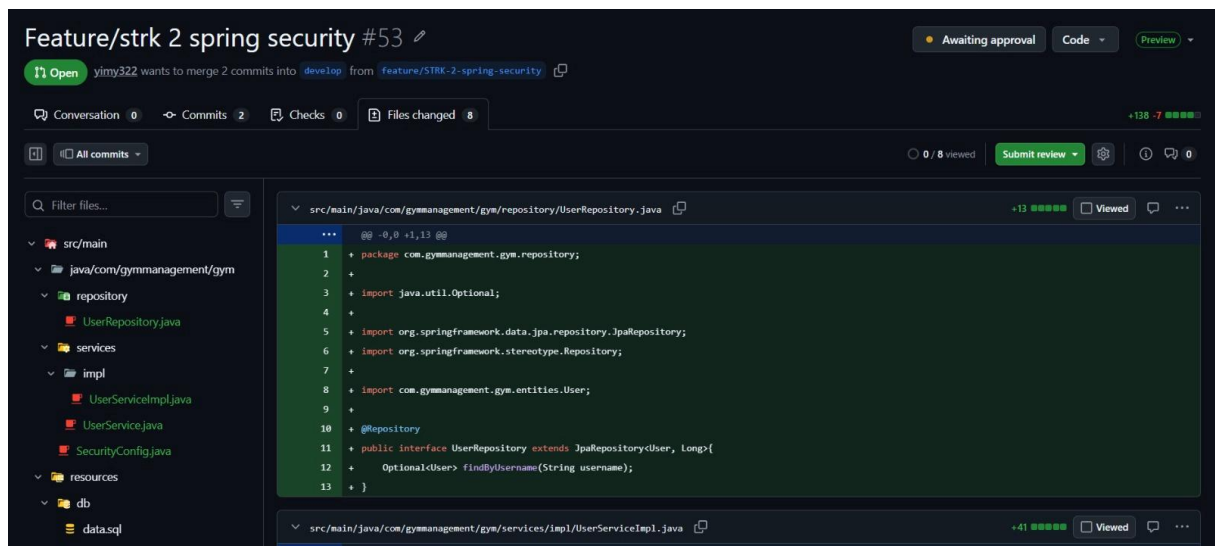
Paso 1. Apertura del Pull Request

Se abre el PR de feature/STRK-2-spring-security hacia develop con 2 commits, quedando en espera de aprobación del revisor.



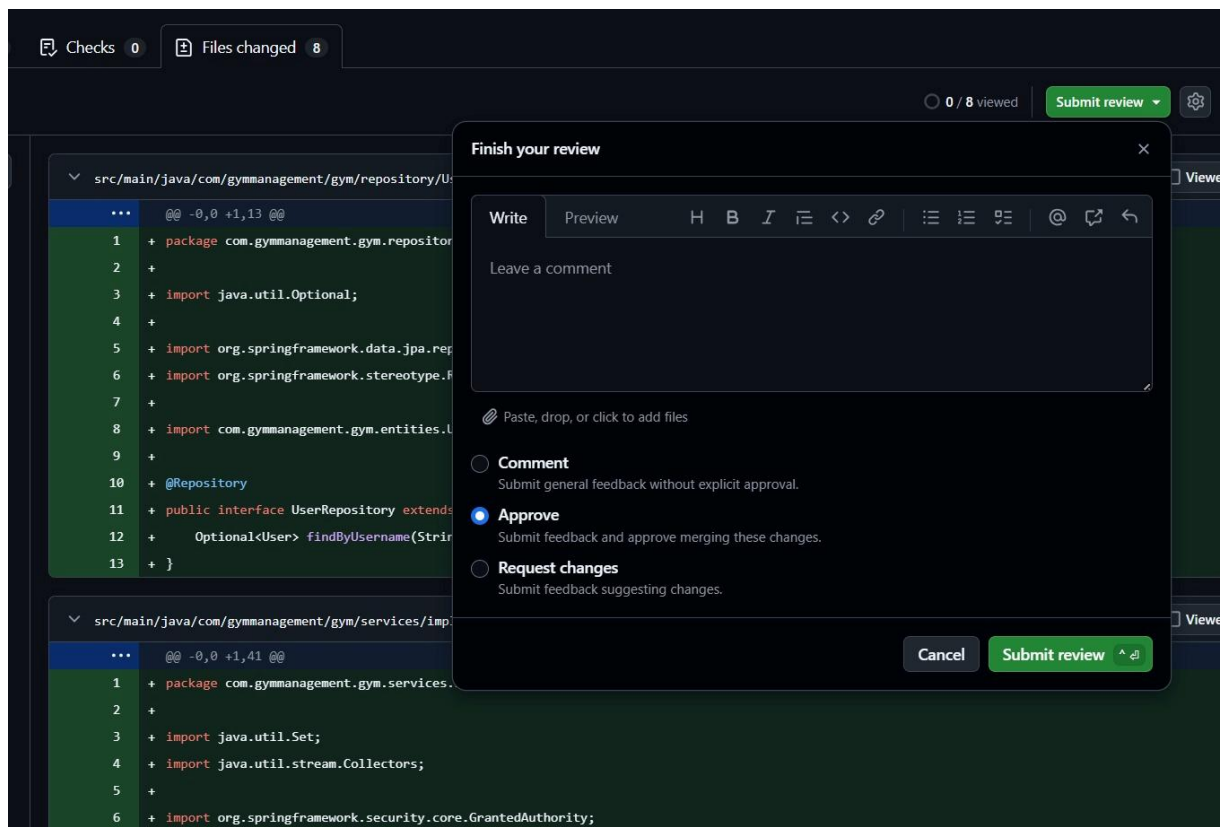
Paso 2. Revisión de archivos

El revisor accede a la pestaña Files changed para revisar los 8 archivos modificados.



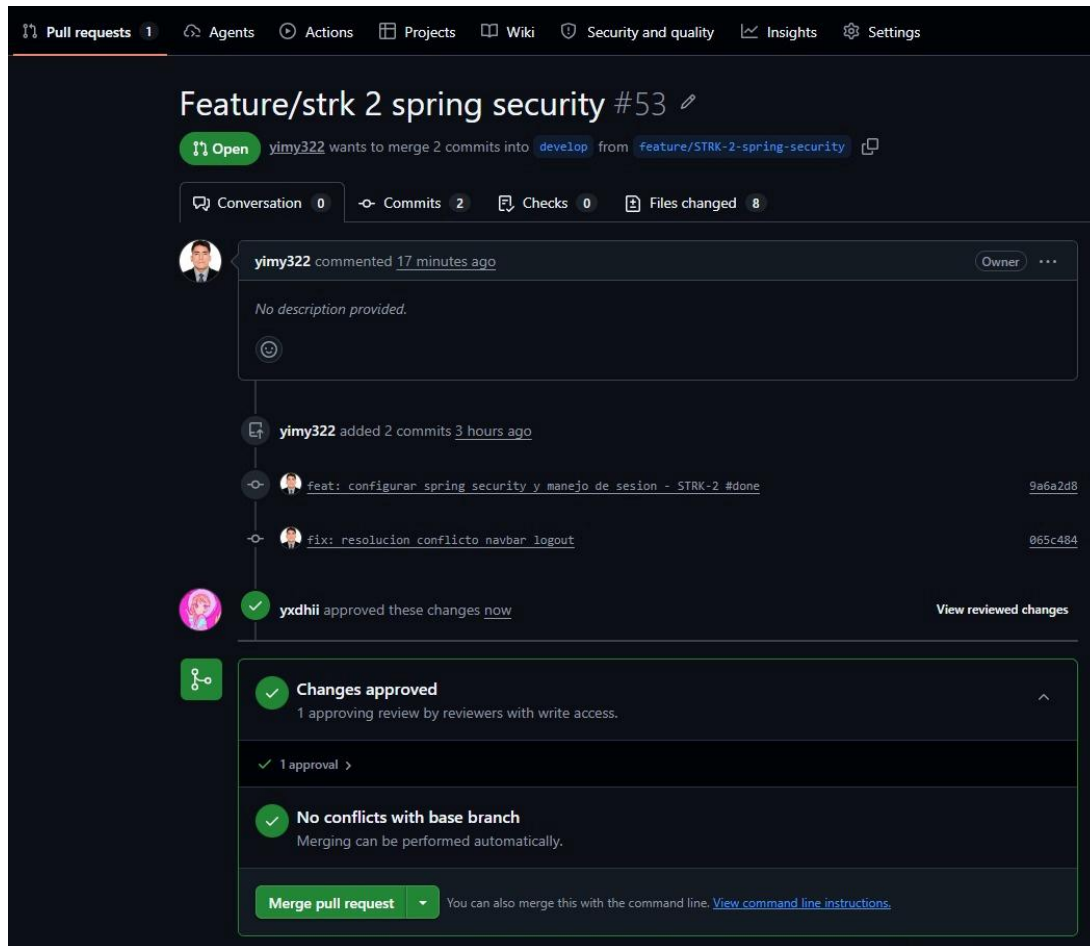
Paso 3. Aprobación

Tras revisar los archivos, el revisor selecciona Approve y hace clic en Submit review para aprobar el PR.



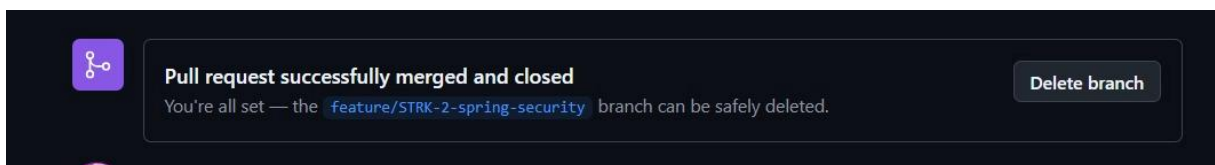
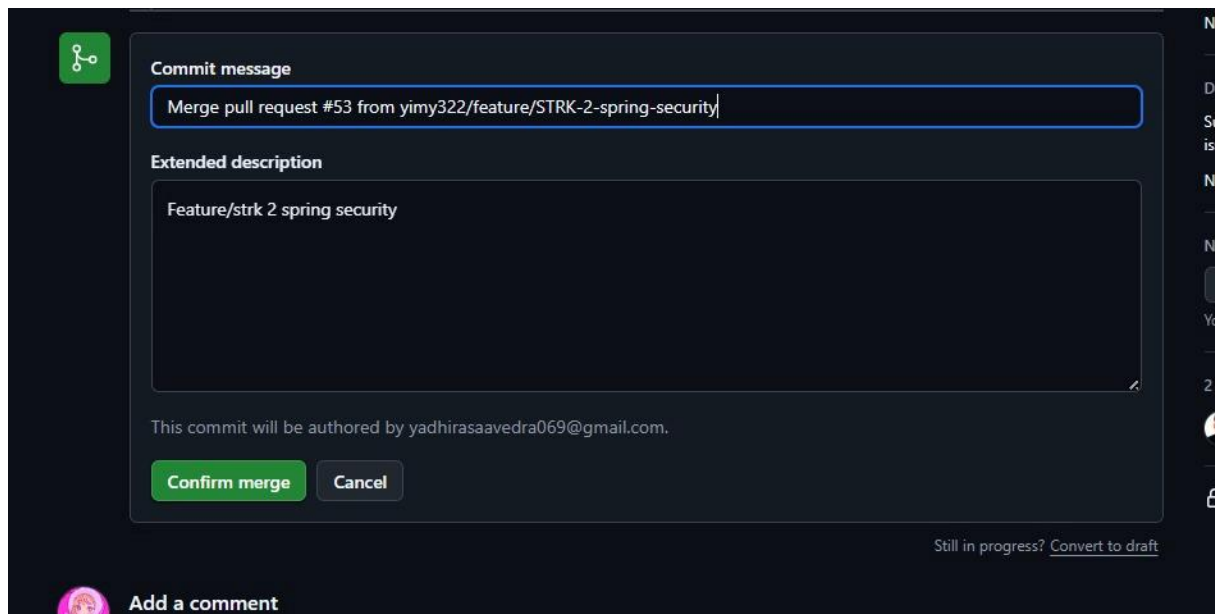
Paso 4. Merge

Una vez aprobado, el PR muestra Changes approved y No conflicts with base branch, habilitando el botón para realizar el merge.



Paso 5. Confirmación

Se confirma el merge con Confirm merge, fusionando la rama en develop exitosamente y cerrando el PR.



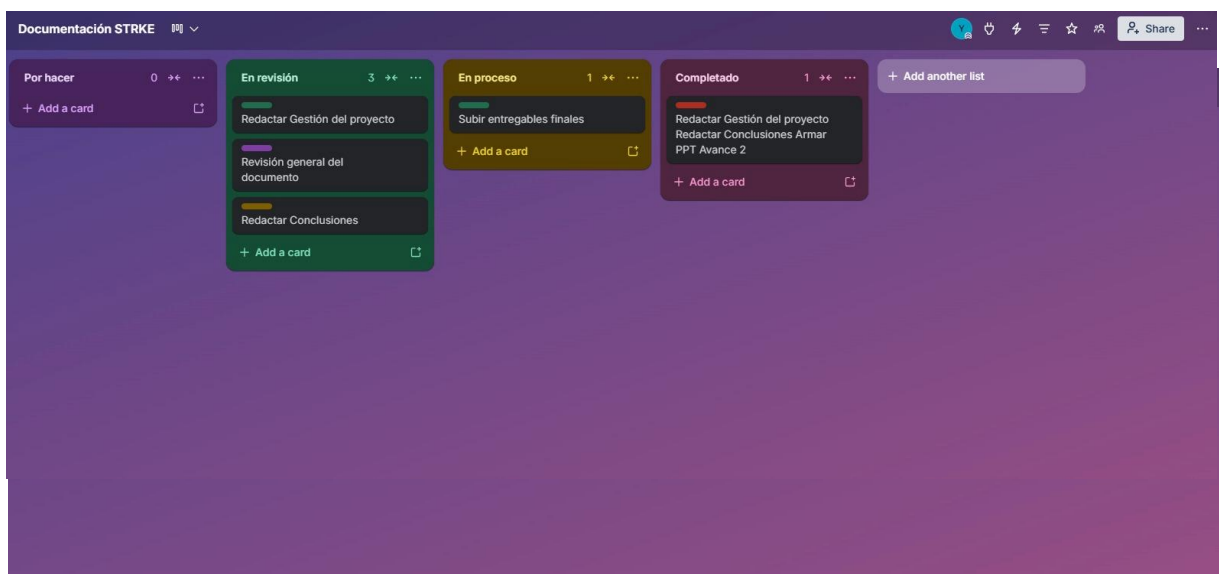
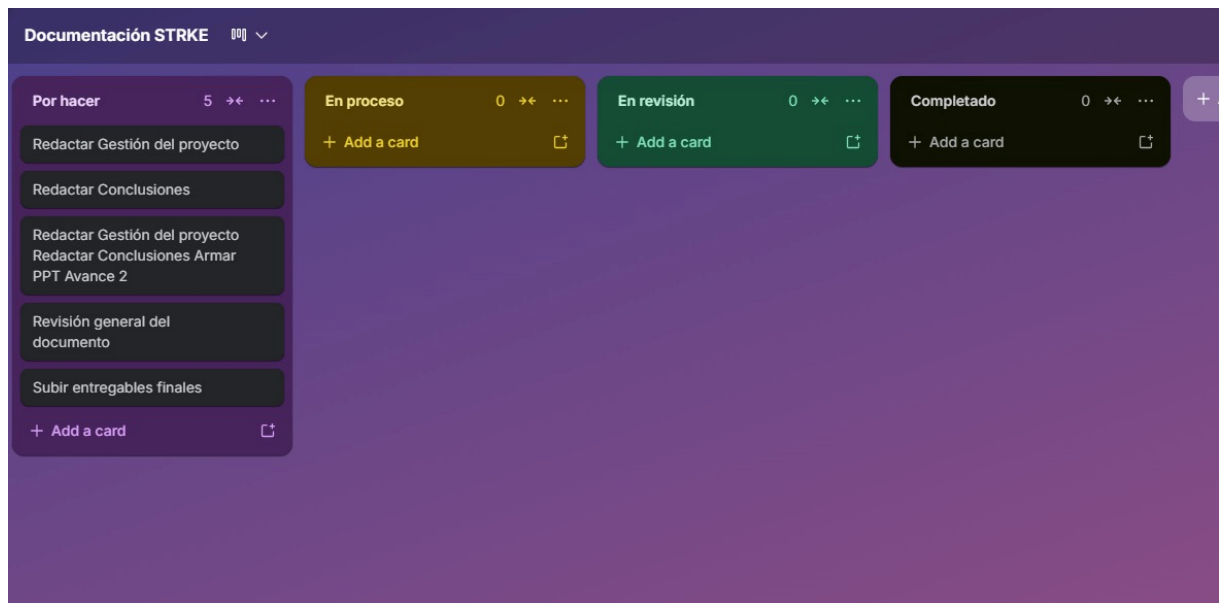
4.11. Herramienta de colaboración (Trello)

Trello es una herramienta de gestión visual de tareas basada en tableros Kanban que permite organizar actividades mediante tarjetas distribuidas en columnas según su estado. A diferencia de Jira, utilizado para el seguimiento del desarrollo del sistema, Trello se empleó para gestionar las actividades de documentación del proyecto.

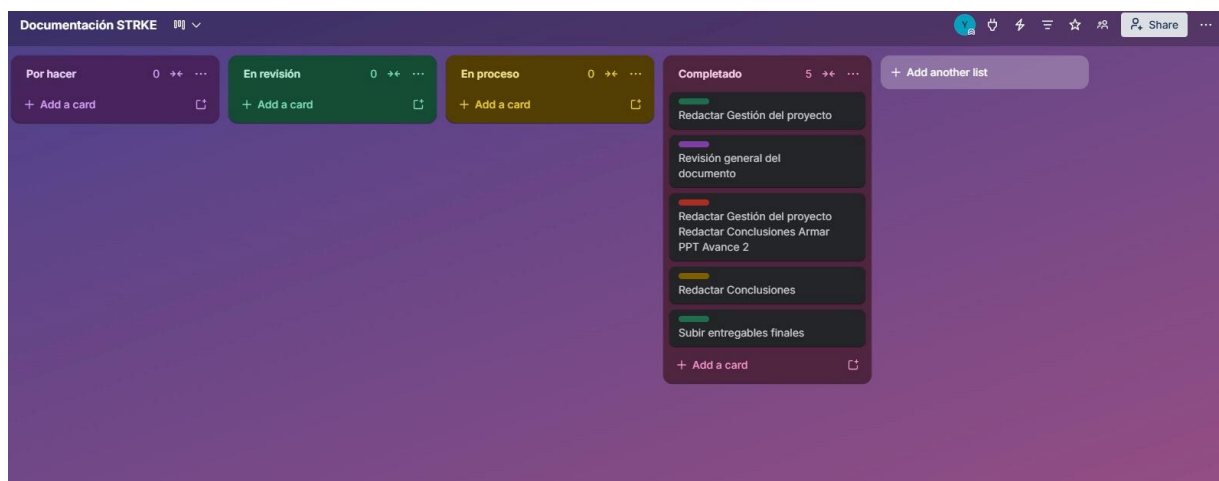
El tablero **Documentación STRKE** se organizó en cuatro columnas: **Por hacer**, **En proceso**, **En revisión** y **Completado**, con las siguientes actividades:

- ❖ Redactar Gestión del proyecto
- ❖ Redactar Conclusiones
- ❖ Redactar Gestión del proyecto, Redactar Conclusiones y Armar PPT Avance 2
- ❖ Revisión general del documento
- ❖ Subir entregables finales

Al cierre del avance, las 5 actividades se encuentran en estado **Completado**.



- *Actividades Completadas*



4.12. Release y Tag

Una vez completado el desarrollo del Sprint 1, se procedió a crear el release oficial del proyecto siguiendo los pasos del flujo GitFlow.

Paso 1. Nos ubicamos en la rama develop ejecutando `git checkout develop`.

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git checkout develop
Already on 'develop'
Your branch is up to date with 'origin/develop'.
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$
```

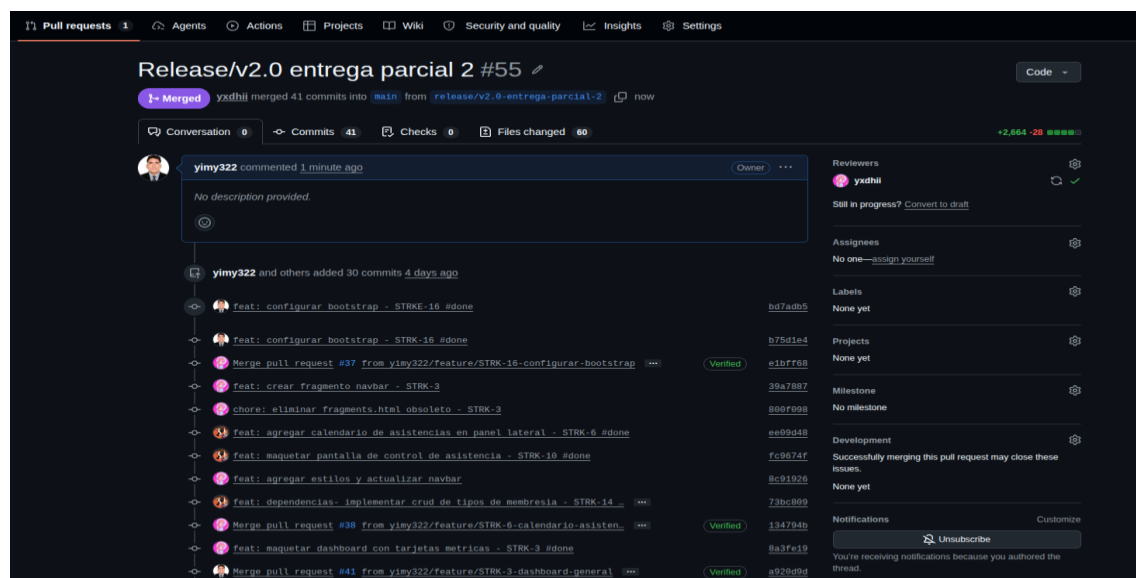
Paso 2. Se crea la rama del release ejecutando `git checkout -b release/v2.0-entrega-parcial-2`.

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git checkout -b release/v2.0-entrega-parcial-2
Switched to a new branch 'release/v2.0-entrega-parcial-2'
```

Paso 3. Se sube la rama al repositorio remoto ejecutando `git push -u origin release/v2.0-entrega-parcial-2`.

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git push -u origin release/v2.0-entrega-parcial-2
Username for 'https://github.com': yimy322
Password for 'https://yimy322@github.com':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'release/v2.0-entrega-parcial-2' on GitHub by visiting:
remote:   https://github.com/yimy322/gym-management-system/pull/new/release/v2.0-entrega-parcial-2
remote:
To https://github.com/yimy322/gym-management-system.git
 * [new branch]      release/v2.0-entrega-parcial-2 -> release/v2.0-entrega-parcial-2
Branch 'release/v2.0-entrega-parcial-2' set up to track remote branch 'release/v2.0-entrega-parcial-2' from 'origin'.
```

Paso 4. Se crea el Pull Request de `release/v2.0-entrega-parcial-2` hacia `main`, el revisor aprueba y se realiza el merge exitosamente.

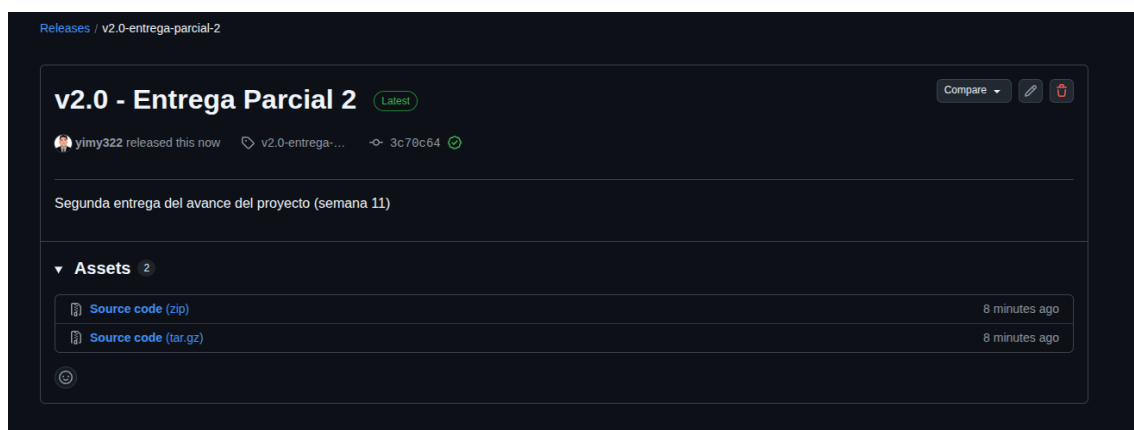
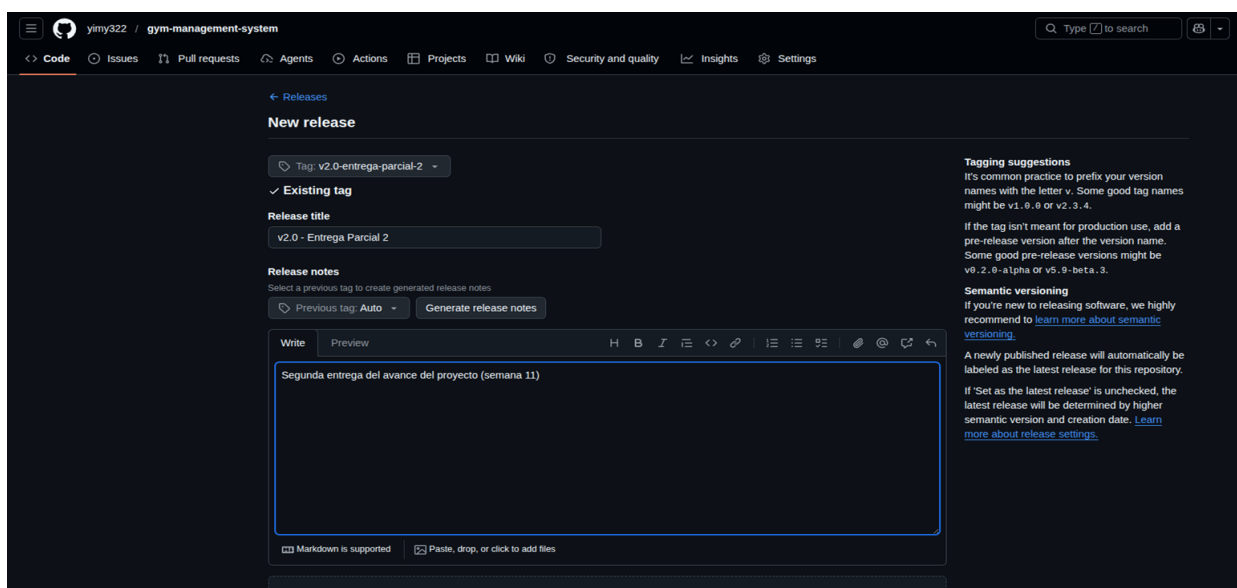


Paso 5. Con el merge aprobado, se crea el tag y se sube al repositorio remoto.

```
Yadhira Saavedra@LAPTOP-22GAEE5Q MINGW64 ~/Documents/STRK
$ git tag -a v2.0-entrega-parcial-2 -m "Avance 2 completo"
$ git push origin v2.0-entrega-parcial-2
```

```
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git tag -a v2.0-entrega-parcial-2 -m "Avance 2 completo"
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$ git push origin v2.0-entrega-parcial-2
Username for 'https://github.com': yimy322
Password for 'https://yimy322@github.com':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 172 bytes | 172.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com:yimy322/gym-management-system.git
 * [new tag]          v2.0-entrega-parcial-2 -> v2.0-entrega-parcial-2
yimy@PC-322:~/universidad/ciclo-8-prt2/herramientas-desarrollo/gym-management-system$
```

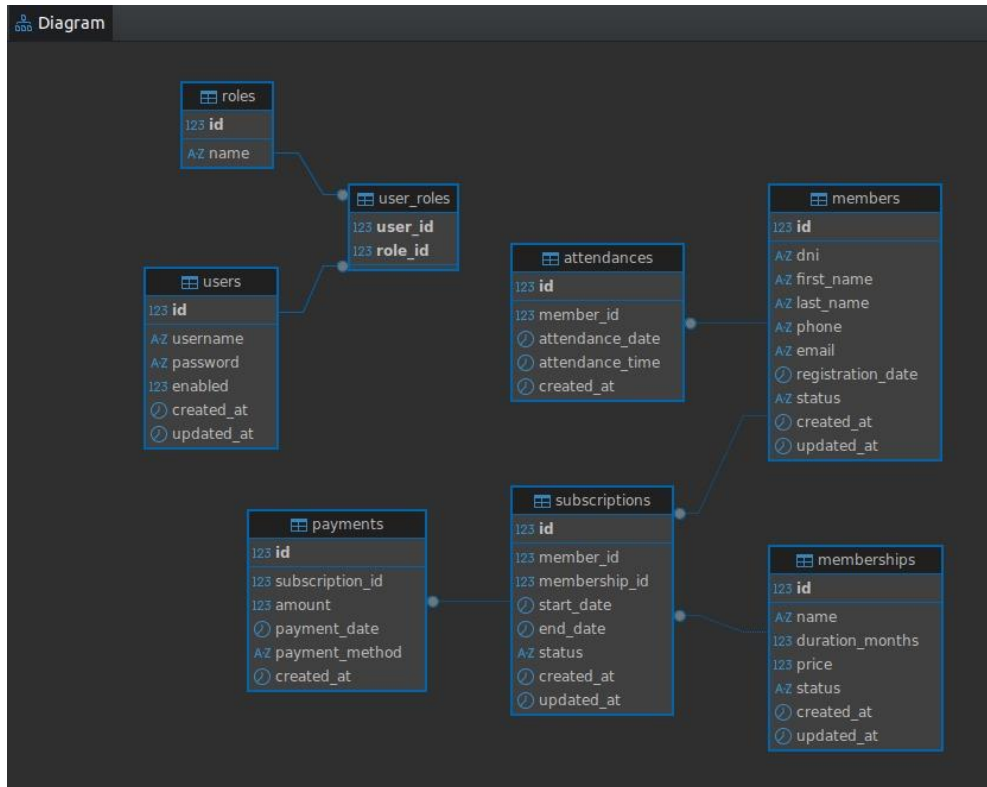
Paso 6. En GitHub se crea el release seleccionando el tag v2.0-entrega-parcial-2, generando el código fuente descargable en .zip y .tar.gz.



CAPÍTULO 5

Diagrama del proyecto

5.1. Base de Datos



Conclusiones

- El desarrollo de este avance permitió al equipo consolidar las bases del trabajo colaborativo mediante el uso de Git y GitHub como herramientas centrales de control de versiones, aplicando el flujo GitFlow con ramas main, develop y feature/ para organizar el desarrollo de forma estructurada y paralela.
- La integración entre GitHub y Jira fortaleció la trazabilidad del proyecto, vinculando cada rama y commit con su issue correspondiente mediante el identificador del ticket, y automatizando el cierre de tareas a través de la palabra clave #done en los mensajes de commit.
- El uso del tablero Kanban en Jira permitió distribuir las actividades de desarrollo de forma equitativa entre los integrantes, hacer seguimiento del avance en tiempo real y mantener visibilidad sobre el estado de cada tarea durante el Sprint 1.
- La gestión de Pull Requests con revisión de código y la resolución documentada del Merge Conflict demostraron la importancia de establecer flujos de trabajo claros y colaborativos, garantizando la calidad del código antes de cada fusión hacia la rama develop.
- El uso de Trello como herramienta complementaria permitió organizar y dar seguimiento a las actividades de documentación del proyecto, manteniendo un control visual del avance de cada entregable hasta su completado.